EXPLORATORY DATA ANALYSIS

CHAPTER 10

Visualizing Distributions

Choosing, constructing, and interpreting visual displays of numerical data

Focus: histograms, KDE, ECDF, boxplots, violin plots, strip/swarm plots, rug plots, group comparisons, and visual design.

Technical convention: Matplotlib object-oriented API only; no pyplot global-state plotting calls. Seaborn is used through axes-level functions.

Learning Objectives

- Choose a distribution plot based on the analytical question and variable type.
- Construct and interpret histograms while understanding bins, normalization, and density.
- Use KDE plots responsibly and explain bandwidth and boundary limitations.
- Read ECDFs, boxplots, violin plots, strip plots, swarm plots, and rug plots.
- Compare groups with common scales and appropriate visual encodings.
- Design readable plots with honest axes, labels, legends, annotations, and sample-size context.
- Combine visual evidence with quantitative summaries without overclaiming.

Review of Earlier Chapters

Chapter 1 — What Is Exploratory Data Analysis?

EDA is iterative investigation, not a plotting checklist. Every visual should answer a question; separate observations from explanations.

Chapter 2 — Understanding the Dataset

Know the observational unit, variable meaning, measurement scale, and granularity before choosing a chart.

Chapter 3 — Data Quality Assessment

Missingness, invalid values, duplicates, inconsistent units, truncation, and recording errors can distort visual distributions. Report exclusions.

Chapter 4 — Exploring Numerical Variables

Mean, median, range, variance, SD, quartiles, and IQR provide numerical anchors. Outliers should be investigated, not automatically removed.

Chapter 5 — Exploring Categorical Variables

Category frequencies and proportions matter when distributions are split by group. High-cardinality categories can make legends unreadable.

Chapter 6 — Exploring Date and Time Variables

Durations and event gaps may be highly skewed. A histogram of elapsed time is not a time-series plot of values over calendar time.

Chapter 7 — Descriptive Statistics: Beyond the Basics

Use robust summaries and quantiles alongside plots. SD describes spread around the mean; standard error describes uncertainty in an estimated mean.

Chapter 8 — Understanding Distributions

Histograms depend on bins, ECDFs avoid binning, and KDEs smooth observations using bandwidth. Discrete, bounded, zero-inflated, and rounded data need special care.

Review of Chapter 9 — Distribution Shape: Skewness, Kurtosis, and Quantiles

- Moment skewness describes asymmetry but is sensitive to extremes; near-zero skewness does not prove symmetry.
- Bowley skewness uses Q1, median, and Q3 and is more resistant to extreme endpoints.
- Pearson kurtosis has a normal reference of 3; excess kurtosis has a normal reference of 0. Kurtosis is not simply peak height.
- Quantiles locate observations; IQR and percentile spreads describe selected parts of a distribution. High quantiles require adequate sample sizes.

Chapter 9 Assignment and Project — Discussion Guide

No assignment submission was included, so this is an expected-reasoning guide, not grading of your answers.

- For [2, 3, 3, 4, 4, 5, 5, 6, 7, 8, 9, 12, 18, 35], the upper values extend beyond the central cluster; a right-tail pattern is plausible and should be checked visually.
- Mean versus median offers a clue, but histogram and ECDF should show whether asymmetry is broad or driven by a few extremes.
- Bowley skewness may be less affected by 35 than moment skewness because it focuses on quartiles.
- Report sample size, missingness, and the Pandas/SciPy estimator conventions.

Expected project learning: connect numerical shape measures to empirical visual structure and explain where they agree or differ.

Preview of This Chapter

The goal is not “Which plot looks attractive?” but “Which visual encoding makes the relevant feature easiest to see without hiding important information?”

1. Match the Plot to the Question

Question	Useful plot(s)	Strength / caution
Where are values concentrated?	Histogram, KDE	Histogram shows counts; KDE smooths and depends on bandwidth.
What proportion is below a threshold?	ECDF	Direct cumulative reading; less emphasis on local peaks.
How do groups differ in median and spread?	Boxplot, violin + points	Boxplot is compact; violin shows smoothed shape.
Are there clusters, gaps, repeated values?	Histogram, strip, rug	Overplotting can hide observations.
How do many groups compare?	Small multiples, boxplots	Keep scales consistent.
What are the actual observations?	Strip/swarm plot	Can become cluttered for large samples.

2. Histograms

A histogram partitions the numerical axis into bins and counts observations in each interval. Adjacent bars represent adjacent intervals; unlike a categorical bar chart, numerical bin order and width matter.

- Bin width controls resolution: wide bins can hide structure; narrow bins can create noisy apparent peaks.
- Bin edges matter: shifting them can change apparent peaks and gaps.
- Use domain-informed widths when thresholds matter; otherwise compare a few defensible choices.
- State whether the vertical axis is count, proportion, or density.

For equal-width bins, density = count / (n × bin width), so total bar area is one. With unequal-width bins, height must account for each bin width for area to represent probability.

```
fig, ax = plt.subplots(figsize=(7, 4))
values = df["amount"].dropna()
ax.hist(values, bins="fd", edgecolor="white")
ax.set(title="Distribution of transaction amounts",
       xlabel="Amount (currency units)", ylabel="Count")
fig.tight_layout()
```

Matplotlib’s “fd” option uses the Freedman–Diaconis rule as a binning heuristic, not an objectively correct choice for every dataset.

3. Counts, Proportions, and Density

Count histograms preserve observation counts and are useful when sample sizes are comparable. Proportion histograms show the share per bin. Density histograms normalize area to one, helping compare shapes across different sample sizes. A density value is not the probability of one exact continuous value; probability over an interval is represented by area.

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(values, bins=20, density=True, alpha=0.75,
        edgecolor="white")
ax.set(title="Density histogram", xlabel="Amount", ylabel="Density")
fig.tight_layout()
```

4. Kernel Density Estimation (KDE)

KDE estimates a smooth density by placing a kernel around each observation and adding contributions. Bandwidth controls smoothness: too small can create many bumps; too large can conceal meaningful structure.

```
fig, ax = plt.subplots(figsize=(7, 4))
sns.kdeplot(data=df, x="amount", bw_adjust=1.0,
            fill=True, ax=ax)
ax.set(title="KDE of transaction amounts", xlabel="Amount",
       ylabel="Estimated density")
fig.tight_layout()
```

- KDE is a smoothed estimate, not the raw empirical distribution.
- Try a range of bandwidths when modality or local structure matters.
- Ordinary KDE can leak beyond natural boundaries such as zero.
- KDE can mislead for discrete variables with few unique values, small samples, or heavy rounding.
- Area normalization means peak height is not a count.

5. Empirical Cumulative Distribution Function (ECDF)

The ECDF at x is the proportion of observed values less than or equal to x. It is a step function and requires neither bin widths nor smoothing.

```
fig, ax = plt.subplots(figsize=(7, 4))
```

```
sns.ecdfplot(data=df, x="amount", ax=ax)
ax.set(title="Empirical cumulative distribution", xlabel="Amount",
       ylabel="Proportion at or below x", ylim=(0, 1))
fig.tight_layout()
```

Read the vertical coordinate at a chosen x to estimate the share at or below that value. Read the x coordinate at a chosen cumulative proportion to estimate a quantile. Ties produce vertical jumps. ECDFs are useful for comparing groups and threshold questions.

6. Boxplots

A conventional Tukey boxplot displays Q1, median, Q3, and whiskers extending to the most extreme observations within 1.5 × IQR of the quartiles. Points beyond whiskers are potential outliers, not automatic errors.

```
fig, ax = plt.subplots(figsize=(7, 4))
sns.boxplot(data=df, x="segment", y="amount", ax=ax)
ax.set(title="Transaction amounts by segment",
       xlabel="Segment", ylabel="Amount")
fig.tight_layout()
```

- The box shows the middle 50%; the line is the median.
- Whiskers are not necessarily the minimum and maximum.
- Boxplots compress shape and can hide modes, clusters, and gaps.
- With very large samples, plotting every outlier can clutter the display.

7. Violin Plots

A violin plot mirrors a KDE around a central axis. Width represents estimated density. It can reveal concentrations that a boxplot hides, but inherits KDE bandwidth and boundary limitations.

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.violinplot(data=df, x="segment", y="amount",
               inner="quart", cut=0, ax=ax)
ax.set(title="Distribution by segment", xlabel="Segment",
       ylabel="Amount")
fig.tight_layout()
```

cut=0 limits the drawn KDE extent to the observed range; it does not eliminate boundary bias. Violin width should not be read as sample count unless scaling explicitly encodes count.

8. Strip Plots and Swarm Plots

A strip plot shows individual observations with jitter to reduce overlap. A swarm plot adjusts positions to avoid overlap, but can become slow or crowded for large samples.

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.stripplot(data=df, x="segment", y="amount", jitter=0.25,
              alpha=0.45, size=3, ax=ax)
ax.set(title="Individual observations by segment",
       xlabel="Segment", ylabel="Amount")
fig.tight_layout()
```

These plots reveal ties, clusters, and sparse regions in small or medium samples. For large data, use a documented sample, transparency, or a complementary summary plot.

9. Rug Plots

A rug plot places a tick for each observation along an axis. It can be combined with a histogram or KDE to show where raw observations occur. With thousands of observations, ticks merge into a dark band.

```
fig, ax = plt.subplots(figsize=(7, 4))
sns.histplot(data=df, x="amount", bins="fd", stat="density", ax=ax)
sns.rugplot(data=df, x="amount", height=0.04, ax=ax)
ax.set(title="Density histogram with rug", xlabel="Amount",
       ylabel="Density")
fig.tight_layout()
```

10. Overlaying Visualizations

Overlaying a KDE on a density histogram connects a binned empirical view with a smoothed estimate. Keep scales compatible and avoid too many layers. Do not overlay a count histogram with a density curve unless the curve is rescaled appropriately.

```
fig, ax = plt.subplots(figsize=(7, 4))
sns.histplot(data=df, x="amount", bins="fd", stat="density",
             alpha=0.35, ax=ax)
sns.kdeplot(data=df, x="amount", ax=ax)
ax.set(title="Histogram and KDE", xlabel="Amount", ylabel="Density")
fig.tight_layout()
```

11. Comparing Groups

A fair comparison uses comparable scales and data definitions. Use the same x-axis limits and, where appropriate, the same histogram bins and KDE bandwidth across groups.

```
fig, ax = plt.subplots(figsize=(8, 4))
```

```
sns.ecdfplot(data=df, x="amount", hue="segment", ax=ax)
ax.set(title="ECDF by segment", xlabel="Amount",
       ylabel="Cumulative proportion", ylim=(0, 1))
ax.legend(title="Segment")
fig.tight_layout()
```

- Show group sample sizes; different n affects apparent smoothness.
- Use common bins for side-by-side or overlaid histograms.
- Do not compare independently tuned KDE bandwidths without noting it.
- ECDFs are useful when groups have different sample sizes or quantile questions matter.
- Use small multiples when overlaid colors become hard to distinguish.

12. Small Multiples and Faceting

Small multiples place one plot per subgroup in a shared layout. They reduce overlap and make subgroup-specific structure easier to inspect.

```
g = sns.displot(data=df, x="amount", col="segment",
                col_wrap=3, kind="hist", bins=20, height=3,
                facet_kws={"sharex": True, "sharey": True})
g.set_axis_labels("Amount", "Count")
```

displot is figure-level and returns a FacetGrid, not a Matplotlib Axes. It is useful for multi-panel exploration. Shared axes aid comparison; free scales may reveal within-group structure but weaken magnitude comparisons.

13. Choosing a Plot for Different Data

Data feature	Good starting point	Caution
Continuous, unbounded	Histogram + ECDF; optional KDE	Check bins and bandwidth.
Discrete, few values	Frequency bar chart	KDE may imply nonexistent intermediate values.
Nonnegative, skewed	Histogram + ECDF; boxplot	Do not hide the upper tail.
Bounded, e.g. 0–100	Histogram + ECDF; violin cautiously	KDE may extend beyond valid bounds.
Many groups	Boxplots or small multiples	Preserve common scales.
Small sample	Dot/strip plot + ECDF	Avoid overconfident smooth curves.
Large sample	Histogram, ECDF	Point plots may overplot.

14. Visual Design and Honest Communication

- Give a descriptive title and label axes with units.
- Use a meaningful x-axis range; do not silently crop observations.
- Start count axes at zero when bar length encodes magnitude.
- Use consistent scales for comparisons and state when scales differ.
- Choose accessible colors; do not rely on color alone.
- Use transparency for overlap, but do not obscure dense regions.
- Show sample sizes and disclose filtering, missing-value handling, and sampling.
- Annotate thresholds only when they have domain meaning.
- Keep legends readable and remove redundant decoration.
- Do not call a distribution normal, bimodal, or heavy-tailed based on one fragile visual choice.

15. Reusable Distribution Plotting Helper

This helper creates a histogram and ECDF with Matplotlib's object-oriented API. It returns figure and axes for later customization or saving.

```
def plot_distribution(series, bins="fd", figsize=(10, 4)):
    values = pd.to_numeric(series, errors="coerce").dropna()
    if values.empty:
        raise ValueError("No valid numeric observations.")

    fig, axes = plt.subplots(1, 2, figsize=figsize)
    axes[0].hist(values, bins=bins, edgecolor="white")
    axes[0].set(title="Histogram", xlabel=series.name or "Value",
                ylabel="Count")

    ordered = np.sort(values.to_numpy())
    cumulative = np.arange(1, len(ordered) + 1) / len(ordered)
    axes[1].step(ordered, cumulative, where="post")
    axes[1].set(title="ECDF", xlabel=series.name or "Value",
                ylabel="Cumulative proportion", ylim=(0, 1))
```

```
    fig.tight_layout()
    return fig, axes
```

For a professional toolkit, add options for units, group comparisons, missingness notes, and figure export.

16. Common Mistakes

- Choosing a plot because it is fashionable rather than because it answers a question.
- Changing bins until a desired story appears.
- Comparing histograms with incompatible bins or axes.
- Treating KDE as the true distribution or ignoring bandwidth.
- Using KDE for sparse discrete data without considering assumptions.
- Reading boxplot whiskers as minimum and maximum.
- Assuming violin width encodes sample count.
- Overlaying count and density scales incorrectly.
- Hiding extreme values through unexplained axis limits.
- Comparing groups without showing sample sizes.

Chapter Summary

- Histograms show binned counts, proportions, or densities; bins and normalization matter.
- KDE gives a smooth estimate but depends on bandwidth and can behave poorly near boundaries.
- ECDFs show cumulative proportions and make quantile and threshold comparisons direct.
- Boxplots summarize median, quartiles, and potential outliers compactly but hide detailed shape.
- Violin plots reveal smoothed shape; strip and swarm plots reveal individual observations.
- Rug plots show raw data locations; small multiples help compare many groups.
- Use common scales and bins for fair comparisons and report sample sizes.
- Combine visual evidence with quantitative summaries and data context.

Assignment

Part A — Conceptual Questions

- When is a histogram more useful than a KDE? When is an ECDF more useful than both?
- Explain why histogram bin width and bin origin affect interpretation.
- Why does a density histogram have total area one rather than bar heights summing to one?
- What does KDE bandwidth control? Describe under-smoothing and over-smoothing.
- Why can an ordinary KDE mislead for a variable bounded below by zero?
- What information does a boxplot preserve, and what does it hide?
- How does a violin plot differ from a boxplot?
- When would a strip plot be preferable to a violin plot?
- Why should group histograms use common bins?
- Why should sample size be reported with distribution plots?

Part B — Plot Construction

Choose a numerical variable from a dataset you understand. Create each plot and write 2–4 sentences interpreting it:

- Histogram with count scale and a second with density scale.
- ECDF with at least two marked thresholds.
- KDE with at least two bandwidth settings; explain what changes.
- Boxplot and violin plot for a meaningful categorical grouping variable.
- Strip plot for a manageable sample or a documented sample from a larger dataset.
- One small-multiple comparison using common axes.

Part C — Critical Visual Review

Find or construct two plots of the same data that suggest different impressions because of bin width, bandwidth, axis limits, or scaling. Explain which choices are defensible and what each view hides.

Part D — Coding Assignment

- Implement plot_distribution() using only Matplotlib's object-oriented API.
- Add input validation and a clear error for no valid numeric observations.
- Add optional histogram bins and figure size arguments.
- Create a grouped ECDF and small-multiple histogram for a dataset with at least three groups.
- Use common axes and include group sample sizes in labels or captions.
- Save figures with clear filenames and write concise interpretations.

Coding Project — Distribution Visualization Toolkit

Extend the cumulative EDA master notebook with a reusable distribution-visualization section. Integrate it with Chapter 9 shape summaries and Chapter 7 descriptive statistics.

- Build reusable plotting functions for histogram, ECDF, KDE, boxplot, violin, strip, and grouped comparisons.
- Use Matplotlib's object-oriented API for direct plots; pass explicit Axes to Seaborn axes-level functions.
- Add configurable titles, axis labels, units, bins, bandwidth, and figure dimensions.
- Support grouped comparisons with consistent bins and scales.
- Include small multiples for datasets with several groups.
- Document when each plot is unsuitable, especially for discrete, bounded, sparse, or very large datasets.
- Create examples from at least two datasets or variables.
- Write a short report combining visual evidence with n, median, IQR, selected quantiles, skewness, and excess kurtosis.

Expected reusable components: plot_histogram(), plot_ecdf(), plot_distribution(), and grouped_distribution_plots(). Keep functions composable and return figure/axes objects where appropriate.

Deliverables: updated cumulative EDA master notebook, saved plots, reusable plotting functions, and a concise visual interpretation report.

Preview of Next Chapter

Chapter 11 — Visualizing Relationships

We'll move from one-variable distributions to relationships between two variables. Topics include scatterplots, jitter and overplotting, hexbin and 2D density plots, joint plots, pair plots, categorical–numerical comparisons, and visualizing nonlinear patterns. We'll focus on choosing encodings that reveal association without implying causation.